Flow-based surrogate models for particle tracking

Matthias Remta
CERN, Geneva 1211, Switzerland and
University of Vienna, Vienna 1090, Austria

Yann Dutheil and Francesco Velotti
CERN, Geneva 1211, Switzerland

(Dated: August 24, 2026)

Particle tracking is a fundamental tool for particle-accelerator design and optimisation. Conventional tracking routines provide high accuracy but are computationally demanding, especially when simulating large particle ensembles or long time spans. As a result, optimising moderate- to high-dimensional parameter spaces is challenging, and real-time surrogate models remain out of reach for many applications.

This contribution introduces a surrogate-modelling approach based on conditional flow matching (CFM). A CFM model is trained on tracking simulations of CERN's Proton Synchrotron (PS) over a 10-dimensional parameter space. The trained model reproduces final phase-space distributions with a median squared maximum mean discrepancy (MMD^2) of 3×10^{-4} and mean inference time of 0.04 s, a speed-up of three orders of magnitude over conventional tracking.

To capture distribution-dependent dynamics that vanilla CFM cannot represent, we extend the model with cross-attention over the initial particle ensemble (Cross-Attention-CFM) and demonstrate that this extension recovers performance on a space-charge benchmark in the PS, where vanilla CFM degrades.

Finally, we introduce Hybrid-CFM, in which a small number of conventionally-tracked particles are used to inform the model. On the same 10-dimensional PS task, Hybrid-CFM with 100 auxiliary particles trained on 200 distributions matches the vanilla CFM trained on 1500, and improves the worst-case (90th-percentile) MMD^2 by roughly a factor of four, substantially reducing the upfront cost of building a surrogate.

I. INTRODUCTION

A central objective in accelerator physics is the control of the phase-space distribution of the particle beam. Containing the particles within the aperture of the machine is essential for safe and efficient operation, while more specific requirements on the distribution arise in many situations, such as efficient beam transfer, loss minimisation, focusing at collider interaction points, and the protection of fixed targets [30]. Consequently, simulating the time-evolution of particle distributions is essential for the design, operation and optimisation of modern particle accelerators. In practice, such simulations represent the distribution as a large ensemble of macro-particles and evolve them with particle-tracking routines based on symplectic integration [11, 17, 20]. These routines provide high accuracy but are computationally demanding, especially when simulating long time spans, large ensembles, or many parameter configurations. As a result, design studies, optimisation campaigns and real-time control loops are bottlenecked by tracking cost.

Three approaches have been developed to alleviate this cost. *Differentiable tracking codes* [16, 20] enable gradient-based optimisation through auto-differentiation. This approach is highly effective for linear or short-section problems but becomes intractable for long-term tracking in circular accelerators, where the computation graph grows prohibitively with the number of turns. *Black-box neural surrogates*, including DeepONet and Fourier neural-operator architectures, have been used to predict beam-envelope statistics and downstream quantities in linear accelerators [9]. These models offer large inference

speed-ups but struggle with non-linear dynamics for long integration times [31]. *Structure-preserving surrogates* enforce symplecticity in the architecture. Examples are physics-based networks built on the Taylor expansion of the transfer map [18], Hénon-map architectures [3, 15] and SympNet [19, 31]. The strong inductive bias improves accuracy on single-particle dynamics, but iterative turn-by-turn inference limits speed-ups.

What none of these threads addresses simultaneously is (i) modelling the full six-dimensional phase-space distribution – with or without collective effects – rather than per-particle or low-dimensional summary maps, (ii) end-to-end differentiability with respect to high-dimensional machine settings at long-term-tracking scales and (iii) sub-second inference. We close this gap with a generative surrogate based on *conditional flow matching* (CFM) [24, 38]. CFM learns a continuous-time vector field that maps an initial phase-space distribution to the final distribution after tracking, conditioned on the machine settings.

The contributions of this work are threefold:

1. We introduce CFM as a surrogate for particle tracking conditioned on machine parameters, and demonstrate near-real-time inference on a 10-dimensional task in the PS.

2. We extend the model with cross-attention over the initial particle ensemble (CA-CFM) to capture distribution-dependent dynamics that vanilla CFM cannot represent, recovering performance on a space-charge problem.

3. We introduce *Hybrid-CFM*, in which a small number of conventionally-tracked auxiliary particles is cross-attended to inform the model, achieving up to a 7.5× reduction in upfront tracking-simulation cost at matched median accuracy and up to a 4× improvement in worst-case performance.

Section II introduces CFM, the CA-CFM and Hybrid-CFM extensions, the network architecture, training pipeline and evaluation metric. Section III presents three experiments: a 10-dimensional optimisation task in the PS, a space-charge benchmark, and a data-efficiency study. Section IV discusses outlier behaviour, compute trade-offs and limitations, and Section V concludes.

II. METHODS

A. Conditional flow matching

Flow matching (FM) [24] learns to transform samples from a simple initial distribution p_0 into samples from a target distribution p_1 by integrating a time-dependent vector field $v_t : \mathbb{R}^d \to \mathbb{R}^d$ over $t \in [0, 1]$. The field v_t generates a flow g_t via

$$\frac{d}{dt} g_t(x) = v_t(g_t(x)), \quad g_0(x) = x, \tag{1}$$

which pushes p_0 forward to $p_t = [g_t]_* p_0$ through the change-of-variables formula. A neural network $u_t(x; \theta)$ is trained to match v_t by minimising

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathop{\mathbb{E}}_{t,\, p_t(x)} \|u_t(x; \theta) - v_t(x)\|^2. \tag{2}$$

However, neither p_t nor v_t is available in closed form in general, rendering $\mathcal{L}_{\mathrm{FM}}$ intractable [24]. Conditioning on sample pairs (x_0, x_1) drawn from a joint distribution $q(x_0, x_1)$ yields the tractable conditional FM (CFM) objective [24, 38]

$$\begin{aligned} \mathcal{L}_{\mathrm{CFM}}(\theta) = \mathop{\mathbb{E}}_{t,\, q(x_0,x_1),\, p_t(x|x_0,x_1)} \\ \|u_t(x; \theta) - v_t(x|x_0, x_1)\|^2, \end{aligned} \tag{3}$$

which shares the same θ-gradients as $\mathcal{L}_{\mathrm{FM}}$ [24]. Following Ref. [38], we adopt Gaussian conditional paths

$$p_t(x|x_0, x_1) = \mathcal{N}\big(x \,\big|\, tx_1 + (1 - t)x_0,\, \sigma^2 I\big)\,, \tag{4}$$

with σ being a small fixed hyperparameter (see Appendix B). The corresponding conditional

vector field is the constant

$$v_t(x|x_0, x_1) = x_1 - x_0. \tag{5}$$

We apply CFM to learn the map between initial and final beam distributions in phase space. The endpoints p_0 and p_1 are represented by ensembles $(x_0^{(i)})_{i=1}^N$ and $(x_1^{(i)})_{i=1}^N$ of 6-dimensional phase-space coordinates before and after tracking, respectively. Because the CFM objective does not require paired samples, the particle correspondence between $x_0^{(i)}$ and $x_1^{(i)}$ is discarded during training. We condition the learned vector field on the machine settings via an additional input $c \in \mathbb{R}^{d_c}$, writing $u_t(x; \theta, c)$, so that a single trained model covers the entire parameter space.

B. Cross-Attention-CFM

Vanilla CFM assumes that, for fixed c, the dynamics depend on each particle's initial coordinates but not on the rest of the initial ensemble. This assumption fails for collective effects, where every particle's trajectory depends on the global distribution. For initial distributions parameterised by a small set of scalars this dependence could in principle be absorbed into c. For non-parametric distributions, however, there is no obvious way to do so, and we instead extend the model with cross-attention [39] between the initial ensemble and the network's latent representation. Each initial coordinate $x_0^{(i)}$ is embedded into the network's hidden dimension and serves as a key/value token. The latent representations inside the backbone act as queries. We refer to this architecture as *Cross-Attention-CFM* (CA-CFM). Because no positional encoding is applied, cross-attention is permutation-invariant in the token dimension and naturally handles ensembles of arbitrary and varying size.

C. Hybrid-CFM

In high-dimensional non-linear problems the required volume of training data can become a limiting factor for surrogate models. We mitigate this curse of dimensionality by combining the CFM model with computationally cheap conventional simulations. Concretely, for each query distribution we run conventional single-particle tracking for a small auxiliary ensemble of $N_{\text{aux}} \ll N$ particles drawn from the same initial distribution as the main ensemble, and present the resulting *final* phase-space coordinates $\{x_1^{(\text{aux},j)}\}_{j=1}^{N_{\text{aux}}}$ as cross-attention tokens to the network. We call this architecture *Hybrid-CFM*.

D. Surrogate-modelling pipeline

A surrogate model for a given application is built as follows. We first define a parameter space

$$\mathcal{P} = \prod_{i=1}^{d_c} [a_i, b_i), \tag{6}$$

encoding for example initial-distribution parameters, magnet strengths, and the voltages and frequencies of radio-frequency cavities. We then draw n points from $\mathcal{P}$, using grid sampling for low-dimensional spaces and scrambled Sobol sampling [26] otherwise, which produces a low-discrepancy sequence in the hypercube $[0, 1)^{d_c}$ and covers $\mathcal{P}$ efficiently. For each of the n configurations we perform a conventional tracking simulation, evolving an ensemble of N particles chosen to describe the distribution adequately. The resulting dataset is used to train the CFM model (or one of its extensions). After training, the model can be used in two regimes: (i) as a fast tracking surrogate during forward design studies, and (ii) as a differentiable map from machine settings to the final distribution, enabling gradient-based search for an optimal $c^\star \in \mathcal{P}$ with respect to any differentiable objective function $\mathcal{L}$ that measures the distance to the target distribution. Figure 1 shows a flowchart of the pipeline.

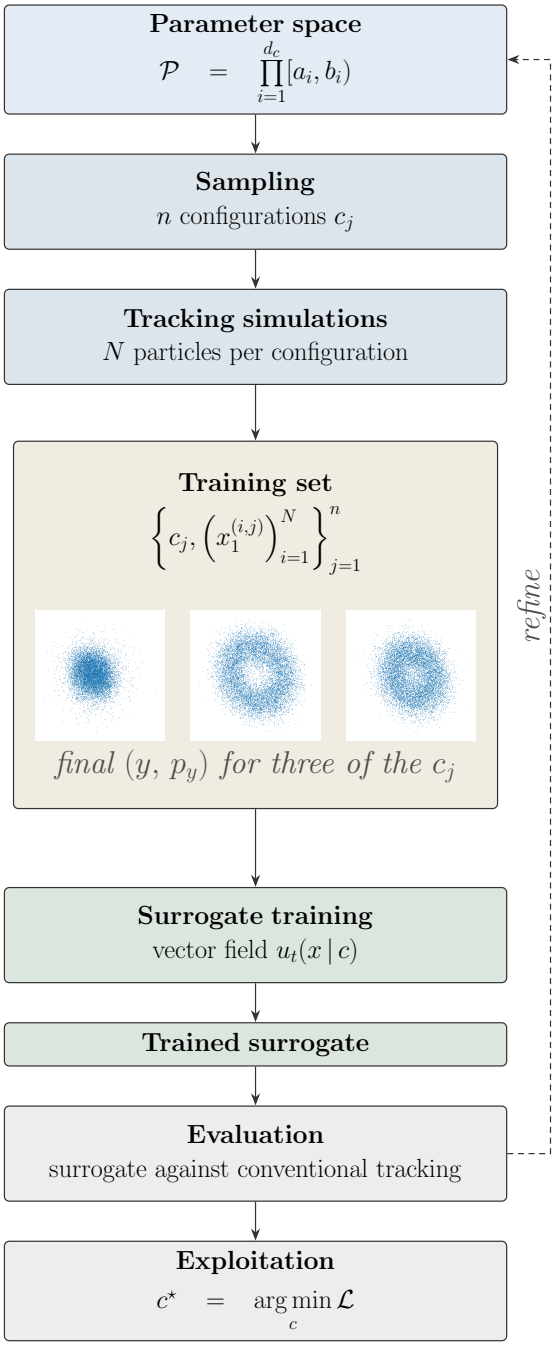

FIG. 1. Flowchart of the surrogate pipeline.

E. Network architecture

The vector field u_t is parametrised by a residual network [12] with ten blocks, each consisting of two fully-connected layers bypassed by a skip connection. The network takes as input the seven-dimensional concatenation of the phase-space coordinate $x \in \mathbb{R}^6$ and the scalar time t, and outputs the corresponding six-dimensional vector field. The conditioning vector $c \in \mathbb{R}^{d_c}$ enters as an additional input to each residual block through a separate channel, in addition to being injected once at the network entry point. For CA-CFM and Hybrid-CFM (Sections II B and II C), cross-attention modules are inserted into residual blocks and operate on the respective token sets. At inference time, samples are drawn by integrating Eq. (1) from $t = 0$ to $t = 1$ with u_t in place of v_t, using an explicit ODE solver. The architecture and inference procedure are visualised in Fig. 2. Detailed hyperparameter values are reported in Appendix A.

F. Evaluation metrics

Because the model is trained without particle correspondence and outputs an ensemble rather than per-particle predictions, we require a metric that compares two distributions directly. We use the squared maximum mean discrepancy (MMD^2) [10]: let $(x)_{i=1}^N$ be a sample from the ground-truth phase-space distribution and $(y)_{i=1}^M$ a sample generated by the surrogate model, then an estimator of MMD^2 is given by

$$\widehat{\mathrm{MMD}}^2(x, y) = \frac{1}{N^2}\sum_{i,j=1}^{N} k(x_i, x_j) - \frac{2}{MN}\sum_{i,j=1}^{M,N} k(x_i, y_j) + \frac{1}{M^2}\sum_{i,j=1}^{M} k(y_i, y_j). \tag{7}$$

We use the radial basis function (RBF) kernel $k(z_1, z_2) = \exp\left(-\gamma\|z_1 - z_2\|^2\right)$ with bandwidth $\gamma = 1/6$. The MMD is a metric on the space of probability distributions with this kernel [10]. Henceforth, we simply write MMD^2 when we

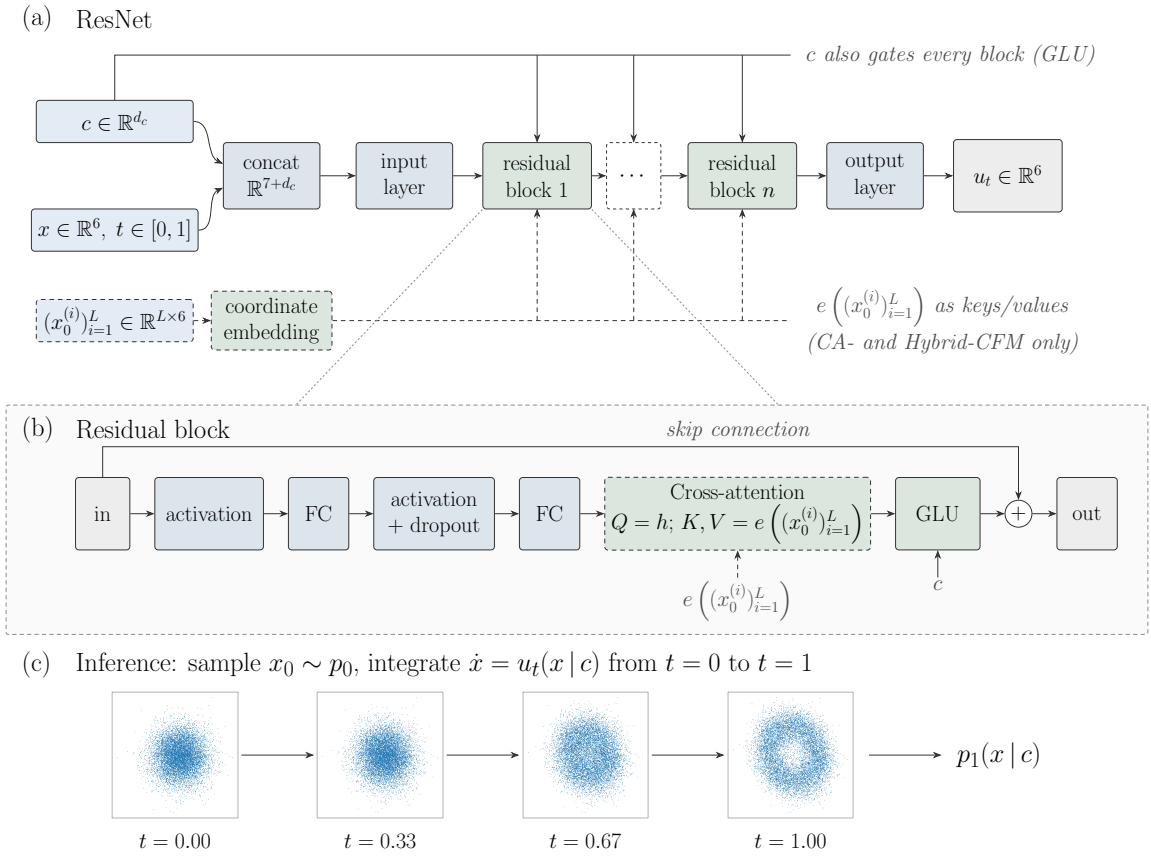

FIG. 2. (a) General layout of the ResNet: the inputs are the phase-space coordinate $x \in \mathbb{R}^6$, the machine settings $c \in \mathbb{R}^{d_c}$ and time $t \in [0, 1]$. CA-CFM and Hybrid-CFM additionally take $(x_0^{(i)})_{i=1}^L \in \mathbb{R}^{L\times 6}$, the phase-space coordinates of L particles, as input. In the case of CA-CFM, these particles are samples from the initial distribution (see Sec. II B) and for Hybrid-CFM, they are the auxiliary particles (see Sec. II C). (b) Layers of each residual block: The input (in) is passed through two interleaved activation and fully-connected (FC) layers. Optionally, a dropout layer [35] follows the second activation layer. In the case of CA-CFM and Hybrid-CFM, cross-attention between the internal representation h and the embedded particle ensemble $e(x_0^{(i)})_{i=1}^L$ is performed afterwards. Finally, a gated linear unit [5] (GLU) injects the machine settings c. A skip connection bypasses these steps, hence the output (out) is the sum of the processed and raw inputs. (c) Inference: sample coordinates x_0 from the initial phase-space distribution p_0 and integrate the vector field u_t from $t = 0$ to $t = 1$ to obtain samples from the final phase-space distribution p_1. We use the Dormand-Prince 5(4) [7] integration scheme.

refer to the estimator given in Eq. 7. As an accelerator-meaningful sanity check, the space-charge experiment of Section III B additionally reports the predicted versus true tail population, which probes whether the model captures the halo content. Here, the tail population is defined as the fraction of particles with action $J > J_{4\sigma}$ where $J_{4\sigma}$ is the action of a particle at an amplitude of $4\,\sigma$.

G. AI-tools

Claude Opus 5 and Claude Fable 5 (both Anthropic) were used to generate, debug, clean and annotate code for the companion repository (https://gitlab.cern.ch/abt-optics-and-code-repository/simulation-codes/cfmtrack.git). Furthermore, these AI-agents were used to prepare, execute and summarise the hyperparameter study (see Appendix A). Study protocols were drafted and approved by the authors before execution, results were carefully checked and replication pipelines are published in the companion repository.

III. RESULTS

A. Single-particle tracking in the PS

We first apply CFM to single-particle tracking in the PS with a 10-dimensional parameter space, which comprises the normalised strengths of three octupole and three sextupole families together with four AC-dipole chirp parameters (number of turns, amplitude, initial and final frequencies). Training data is generated by sampling 1500 points in this space, with $N = 1 \times 10^4$ final-state particles per configuration produced by conventional tracking with Xsuite [17]. The trained model is evaluated on 100 distributions generated from previously unseen settings drawn from the same space. The model produces 1×10^4 particles per test configuration, though it can in principle generate any number of particles.

The MMD^2 distribution over the test set (top-right panel of Fig. 3) has median 3×10^{-4}, with the bulk of configurations below 1×10^{-3} and a long-tailed minority of outliers, likely caused by the highly non-linear relation between machine settings and final distribution in this example. Section III C presents a Hybrid-CFM-based mitigation. The lower triangle of Fig. 3 compares the one- and two-dimensional projections of the model prediction (orange) against the ground truth (blue) for a representative test configuration, which agree across all projections. CFM produces a 10 000-particle ensemble in approximately 40 ms, three orders of magnitude faster than Xsuite (see Table V).

B. Collective effects: space charge

To test CA-CFM (Section II B), we simulate distribution-dependent dynamics by performing particle-in-cell space-charge tracking in the PS with Xsuite [17]. The initial distributions are mixtures of a matched six-dimensional Gaussian and a halo component, the latter defined by polar coordinates $r_x \sim U(2\sigma_x, 3\sigma_x)$ and $\theta \sim U(0, 2\pi)$ in the normalised horizontal phase-space and $(y, p_y, \zeta, \delta) \sim \mathcal{N}(0, \Sigma)$. The polar coordinates are converted into canonical coordinates (x, p_x), which are independent of (y, p_y, ζ, δ). Evenly spaced mixture weights between 0 and 1 yield a one-parameter family of initial distributions. A mixture weight of 0 yields purely the halo component. Conversely, the matched Gaussian beam is obtained with a mixture weight of 1. Each distribution is tracked for 256 turns with 50 space-charge kicks per turn. Of the 98 distributions, 60 are used for training and 38 for testing.

For both models, the true initial particle coordinates serve as the source distribution x_0 at training and as starting points at inference. For CA-CFM, they additionally serve as cross-attention tokens. Performance as a function of mixture weight is shown in the left panel of Fig. 4: vanilla CFM degrades at small mixture weights, while CA-CFM maintains a consistent MMD^2 across the full range. Small mixture weights correspond to a large halo content in the original distribution, where the stronger non-linear dynamics may explain the degraded performance of vanilla CFM. The right panel of Fig. 4 compares the predicted against the true tail population of the horizontal plane. Vanilla CFM fails to accurately predict strongly populated tails. On the other hand, CA-CFM tracks the true tail population well across the entire range. CA-CFM evolves a 10 000-particle ensemble in approximately 0.4 s, three orders of

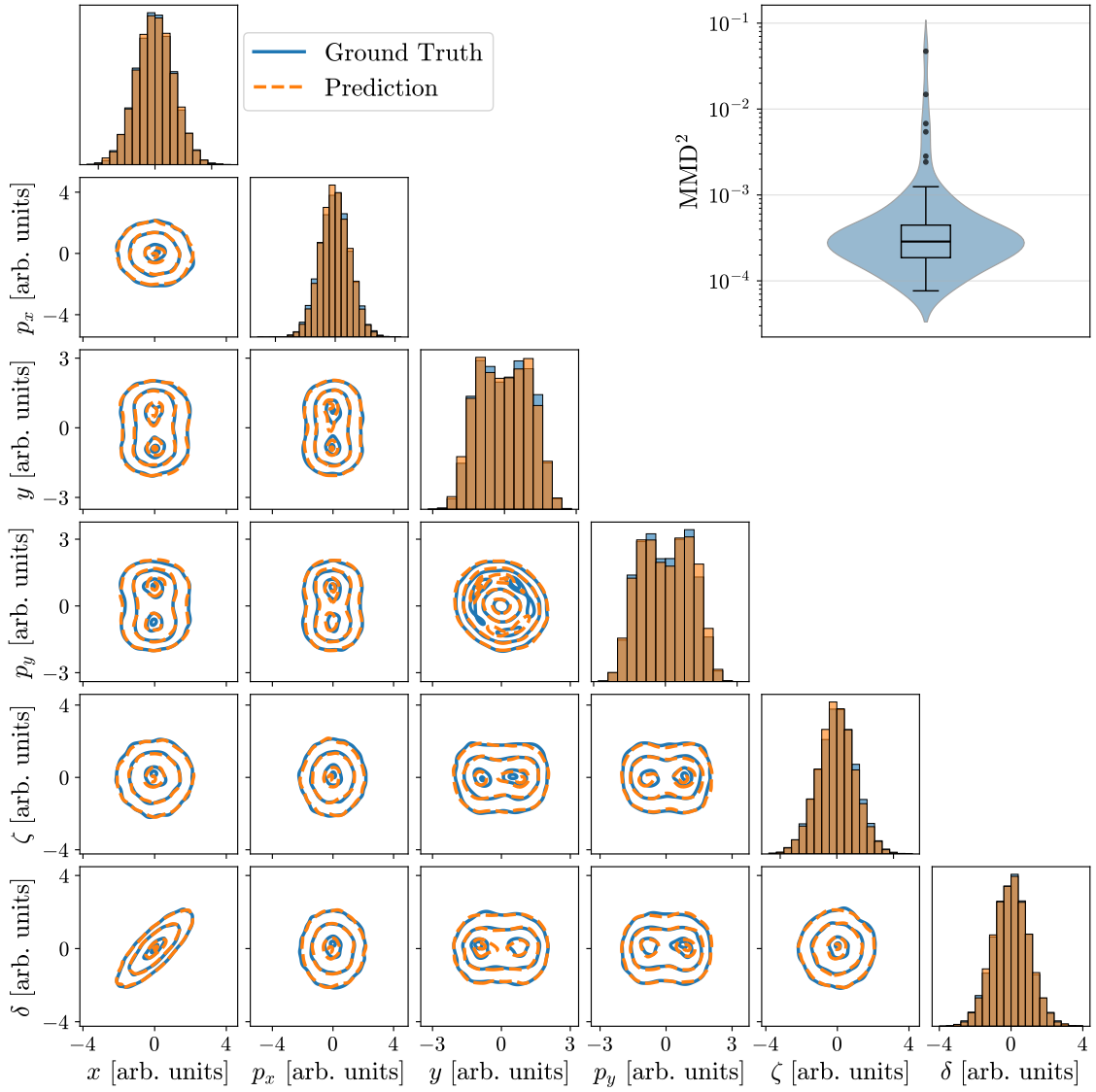

FIG. 3. Single representative test configuration from the 10-dimensional PS task (Section III A). Lower triangle: marginal projections of the six-dimensional phase space, with the CFM prediction (orange) overlaid on the ground truth (blue). Diagonal panels show one-dimensional histograms of each phase-space coordinate. Off-diagonal panels show two-dimensional Gaussian kernel-density-estimation [27] contours, with bandwidth selected by Scott's rule [34]. Top-right inset: histogram of MMD^2 values over all 100 validation distributions. Median MMD$^2 = 3 \times 10^{-4}$, with a long-tailed minority of outlier configurations addressed in Section III C.

magnitude faster than Xsuite (see Table V).

C. Data efficiency with Hybrid-CFM

We evaluate Hybrid-CFM (Section II C) on the same 10-dimensional single-particle PS task as in Section III A, varying both the training-set size and the number of auxiliary particles. As a baseline, we train a vanilla CFM model. Then, we train Hybrid-CFM models with $N_{\text{aux}} \in \{10, 50, 100, 200, 500\}$. Each setting is repeated for training-set sizes of 200, 500 and 1500 distributions ($N = 10\,000$ particles per distribution). The number of training distributions corresponds directly to the upfront tracking-simulation cost, since one conventional tracking run is required per distribution.

Figure 5 summarises the results. The left panel shows MMD^2 for the different models trained on the 200-distribution training set: the vanilla CFM performs well in the median but exhibits a long tail of outlier configurations. With $N_{\text{aux}} = 10$, Hybrid-CFM provides little benefit, presumably because the auxiliary signal is too noisy to anchor the predicted distribution. For $N_{\text{aux}} \gtrsim 50$, however, Hybrid-CFM substantially compresses the outlier tail, with diminishing returns above $N_{\text{aux}} \approx 100$. The right panel shows the 90th-percentile MMD^2 as a function of training-set size for each N_{aux}: Hybrid-CFM improves the worst-case MMD^2 by up to a factor of four. In particular, Hybrid-CFM with $N_{\text{aux}} = 100$ trained on 200 distributions matches the 90th-percentile MMD^2 of the vanilla CFM trained on 1500 distributions – an up to 7.5× reduction in upfront tracking cost at matched worst-case (90th-percentile) accuracy. The inference time of this method for $N = 10\,000$ particles is dominated by the tracking cost of the auxiliary particles and the total speed-up on GPU is negligible (see Table V). Since this auxiliary cost is independent of N, it is amortized as N grows, and we therefore expect increasing gains over conventional tracking. On CPU, a 47× speed-up was achieved with $N_{\text{aux}} = 50$.

IV. DISCUSSION

The surrogate models proposed here treat particle tracking as a distributional map, collapsing the per-turn motion of N particles into one learned transport. This sidesteps the dominant cost of long-term tracking in circular machines, at the price of giving up particle-level resolvability and any explicit physics prior. The remainder of this section discusses the resulting compute trade-offs, the limitations this framing imposes, and future research directions.

The total compute cost of the approach has three components: the upfront cost of generating the training set by conventional tracking, the cost of training the CFM model, and the cost of per-query inference (and, for Hybrid-CFM, of the per-query auxiliary tracking). Pure conventional tracking pays a large cost at every query, whereas the CFM and CA-CFM models amortise this cost upfront and offer near-real-time inference. Whether this trade-off pays off therefore depends strongly on the cost of a single conventional query. For single-pass systems such as linacs or transfer lines, where a particle traverses each element once, direct tracking is already cheap and the upfront investment is unlikely to be recovered. The picture changes for circular machines: tracking many turns and large ensembles of particles makes each query expensive enough that the upfront cost is repaid after comparatively few forward evaluations. Hybrid-CFM trades a smaller upfront tracking cost for a per-query auxiliary-tracking overhead, which is favourable when training data is scarce and the auxiliary cost remains much smaller than tracking the full N particles.

The surrogate is not symplectic by construction. Enforcing symplecticity would require the learned vector field itself to be symplectic, which in the limit reduces to a Hamiltonian neural network operating at the per-particle level – in essence, the class of physics-informed and integrator-based surrogates discussed in Section I. Such methods must explicitly resolve the non-linear, possibly time-dependent motion of every particle over every turn, which becomes

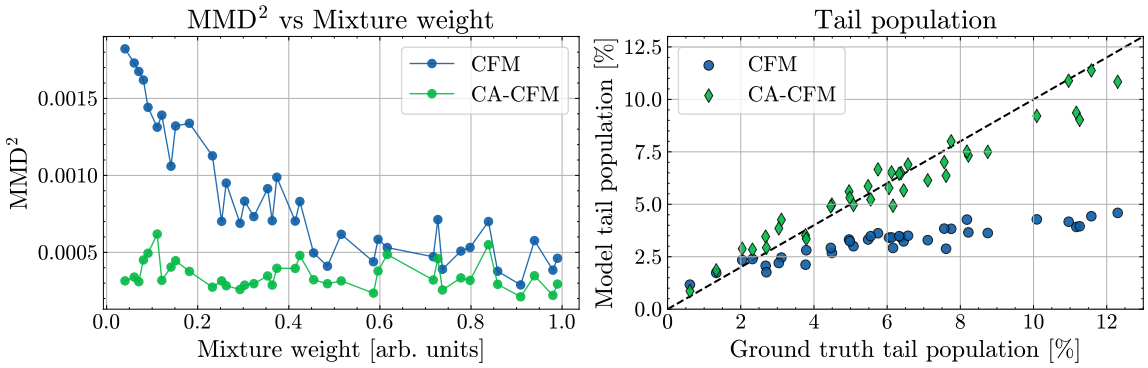

FIG. 4. Left: MMD^2 between the predicted and ground-truth final phase-space distributions for the PS space-charge experiment (Section III B), as a function of the Gaussian–halo mixture weight. Vanilla CFM (blue) degrades at small mixture weights, while CA-CFM (green) maintains a consistent MMD^2 across the full mixture range. Right: model prediction versus ground truth for the fraction of particles in the horizontal tail. The tail population is large due to strongly non-linear dynamics and space-charge effects. Vanilla CFM (blue) systematically underestimates the tails, while CA-CFM (green) agrees well with the ground truth across the full range.

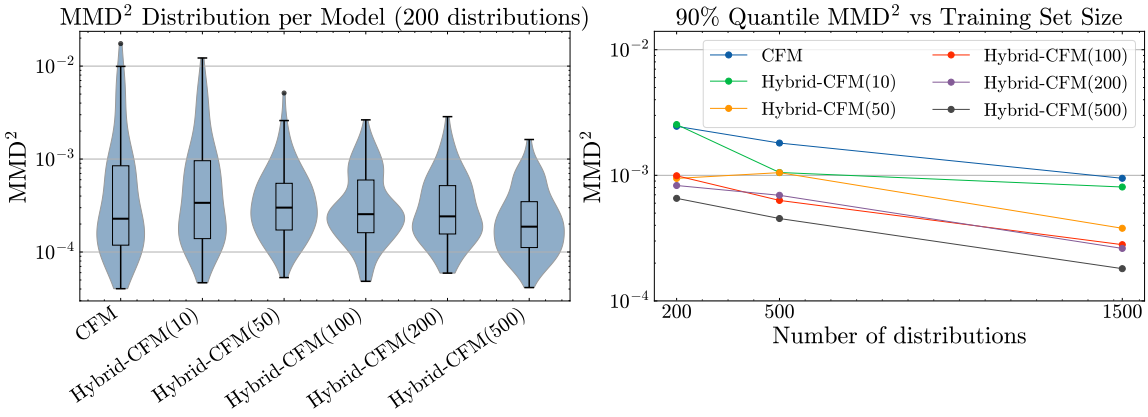

FIG. 5. CFM and Hybrid-CFM models trained with $N_{\mathrm{aux}} \in \{10, 50, 100, 200, 500\}$ auxiliary particles on training sets of 200, 500 and 1500 distributions ($N = 10\,000$ particles per distribution). Left: per-test-configuration MMD^2 for the 200-distribution training set; Hybrid-CFM compresses the outlier tail for $N_{\mathrm{aux}} \gtrsim 50$, with little effect at $N_{\mathrm{aux}} = 10$. Right: 90th-percentile MMD^2 versus training-set size; Hybrid-CFM matches the CFM's 90th-percentile MMD^2 with substantially fewer training distributions and improves the 90th-percentile MMD^2 by up to a factor of four on equally-sized training sets.

the dominant cost for long-term tracking in circular machines [31]. By collapsing that motion into a single distributional map, the present approach sidesteps this bottleneck, but is therefore unsuited to studies that require particle-level accuracy. Another consequence of omitting explicit physics priors is degraded accuracy in regions of the parameter space where training data is sparse. This is visible in the heavy-tailed MMD^2 distribution on the 10-dimensional PS

task in Section III A. Two mitigations are available. The brute-force option is to enlarge the training set, which is expensive because each data-point requires a full tracking simulation. Hybrid-CFM is the more economical alternative: tracking $N_{\text{aux}} \ll N$ auxiliary particles per configuration is cheap, and the resulting tokens anchor the prediction in the physics of the specific configuration. As shown in Section III C, this yields up to a 7.5× reduction in upfront tracking cost at matched worst-case accuracy.

Three further directions deserve mention. First, uncertainty quantification: a downstream optimiser using the surrogate needs to know where the prediction is trustworthy. Model ensembles [22, 25] would address this issue but incur additional training and inference cost. Another promising approach is conformal prediction [1, 21]. Second, incorporation of measured data: instrumentation on real accelerators delivers distribution-level rather than per-particle observables, often as lower-dimensional projections of the full six-dimensional phase space (e.g. two-dimensional screen images, transverse profiles, tomographic reconstructions). The distributional nature of CFM lends itself naturally to such data sources, opening a path to surrogates that combine simulation-based and measurement-based training. Third, system identification [2, 6, 37, 40]: the true machine parameters might be inferred by matching the surrogate output against measured beam distributions. Inverse problems of this kind are inherently iterative — a Bayesian or gradient-based sampler would need to evaluate the forward model many thousands of times while exploring the parameter space — which makes them impractical when each evaluation requires a full tracking simulation. A surrogate that reproduces the tracked distribution in milliseconds could bring such studies within reach, and its differentiability would in principle allow gradients with respect to the machine parameters to be propagated directly to the optimiser.

V. CONCLUSION

We presented a generative surrogate-modelling approach for particle tracking based on CFM. The model is trained on tracking simulations parametrised by machine settings Once trained, it produces final phase-space distributions up to three orders of magnitudes faster than the conventional tracking code. We introduced two extensions: CA-CFM, which captures distribution-dependent dynamics such as collective effects, and Hybrid-CFM, which uses a small ensemble of conventionally tracked auxiliary particles to mitigate the curse of dimensionality in high-dimensional parameter spaces. On a 10-dimensional task in the PS, CFM reproduces final-state phase-space distributions with a median MMD^2 of 3×10^{-4}. Hybrid-CFM achieves matched worst-case accuracy with up to a 7.5× reduction in upfront tracking cost. CA-CFM captures space-charge-induced halo dynamics where vanilla CFM degrades.

Because the surrogate models are differentiable with respect to the machine settings, they can be plugged directly into gradient-based optimisation loops in the parameter space, opening efficient design and online-control workflows that are impractical with conventional tracking. With inference times between 0.04 and 0.4 seconds, the models are fast enough to serve as a building blocks for virtual accelerators and digital twins. The flexibility of the underlying framework – arbitrary conditioning, distribution-level training, and compatibility with conventional tracking methods – suggests broad applicability across accelerator design, optimisation, and online-control tasks.

Beyond particle accelerators, the approach naturally extends to any setting where ensembles of particles evolve under (stochastic) differential equations and the distribution itself is the quantity of interest, such as molecular dynamics and plasma physics.

ACKNOWLEDGMENTS

M.R. designed and carried out the studies, developed the software, and wrote the manuscript. Y.D. and F.V. supervised the work and contributed ideas and feedback in discussions. F.V. contributed in the revision of the manuscript. We acknowledge the substantive usage of AI-tools during this study, as disclosed in Sec. II G.

DATA AND CODE AVAILABILITY

Trained model checkpoints [33] and the simulation datasets [32] used in this work are publicly available on Zenodo. Training and inference code is publicly available on GitLab (https://gitlab.cern.ch/abt-optics-and-code-repository/simulation-codes/cfmtrack.git).

Appendix A: Hyperparameter studies

This section presents the impact of certain hyperparameters on model performance. All hyperparameter studies were performed on the 200-distribution training set from the PS single-particle tracking problem (see Sec. III A). Performance is measured on the validation set (100 distributions), except for the final evaluation, which is performed on the test set (332 distributions).

Network architecture. We obtained our results (see Sec. III) using residual networks [12], which each consist of 10 residual blocks, as backbone of our models. The layout is shown in Fig. 2 and follows the implementation in Ref. [36], where each residual block consists of two non-linear activations (rectified linear unit [14] (ReLU)) interleaved with two linear layers of 128 neurons. Each block is wrapped with a skip connection. Conditioning inputs are injected with a gated linear unit [5] in each block. For our CA-CFM and our Hybrid-CFM, we add a single-head attention block [23, 39] to each residual block. The number of trainable weights is stated in Table V for each model.
We compare this network architecture with two alternatives, sharing all remaining hyperparameters with the published configuration (see Tables II and III): the same network without skip connections, and a plain multi-layer perceptron (MLP) receiving the context once at the input. The three architectures were compared on a grid over depth (2–16 residual blocks / hidden layers at width 128) and width (32–512 at depth 10). Each of the 33 configurations was trained from 20 independent seeds (weight initialisation, data ordering, and path noise all redrawn) — reduced to 15 for the three width-512 cells and to 5 for the five no-skip cells in which all seeds collapse — amounting to 585 runs in total. The MMD^2-performance is shown in Fig. 6. The three architectures perform similarly when the network is shallow (6 or less hidden layers). As the depth is increased, ResNet's performance improves slightly, whilst the MLP degrades and the ResNet without skip connections even collapses. The latter is likely caused by the GLU modulation. Hence, skip connections facilitate deeper architectures, which is consistent with previous studies [12]. On the other hand, the performance curve for different widths is flat, only 32 neurons perform significantly worse across architectures. A ResNet with depth 10 and width 64 provides the same performance as our published baseline with a factor 3.8 less weights.

Activation function. Our baseline architecture uses ReLU as activation function. We compare the performance against Tangens hyperbolicus (Tanh), gaussian error linear unit [13] (GELU) and sigmoid linear unit [8, 13, 29] (SiLU). Only Tanh differs significantly from ReLU (1.34× worse, 95 % confidence interval excludes unity), GELU and SiLU are interchangeable with the baseline. The results are summarised in Table I.

Dropout. Each residual block incorporates a dropout layer (see Fig. 2), which randomly sets input features to zero with a probability q [35]. Our baseline architecture does not use dropout, i.e. $q = 0$. Increasing q for the full-size model (depth 10, width 128) improves the validation MMD^2 monotonically up to $q \approx 0.15$, after which the curve flattens (see Fig. 7).

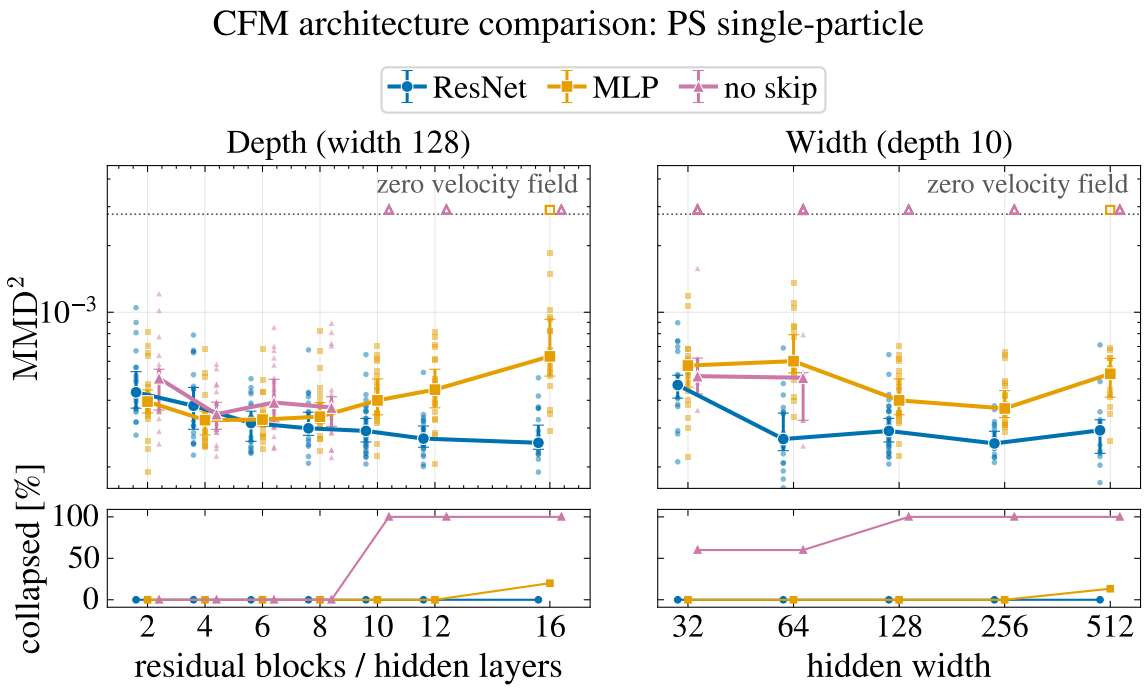

FIG. 6. Three architectures benchmarked on the PS single-particle tracking problem: ResNet (blue), MLP (orange), and the same ResNet without skip connections (magenta). All models are trained on the 200-distribution training set. Each configuration is trained with up to 20 independent seeds, and each seed is scored by the median MMD^2 over 20 configurations from the validation set. Top: markers show the median score over non-collapsed seeds, with 95 % bootstrap confidence intervals on that median. Individual seeds are shown as faint points. Bottom: fraction of seeds that collapsed during training, defined as MMD^2 exceeding the smallest value of 100 evaluations of a model with identically zero velocity field, whose output is simply its initial noise ensemble. Left: width fixed at 128 nodes, depth varied from 2 to 16. Right: depth fixed at 10, width varied from 32 to 512. Markers belonging to the same group are slightly offset horizontally for visual clarity.

At $q = 0.15$ the gain is 0.77 (bootstrapped 95 % confidence interval: $[0.63, 0.96]$) relative to the unregularised model on validation, and 0.84 $[0.80, 0.87]$ on the held-out test set. The gain does not transfer to the smaller model (depth 10, width 64): there, the same dropout leaves the score almost unchanged (gain 0.94 $[0.69, 1.31]$), consistent with a regularisation effect that the reduced capacity no longer requires. Dropout and the reduction in model size are therefore alternatives rather than a combination. Dropout is applied during training only, sampling always uses the deterministic network.

Training. We use AdamW with cosine-annealing as the optimiser. All parameters not listed in Table II are kept at their PyTorch defaults. Batch size and epochs for the published checkpoints are given in Table III. The batch size refers to the number of particles to ensure comparability over all models. For all models, expect CFM on the single-particle task, the particles are grouped per machine configuration into sets of 2000 particles. The number of training epochs depends on the training dataset size, as smaller datasets require more passes to converge. We did not perform early-stopping.

TABLE I. Activation function comparison at the published architecture (10×128 ResNet, all other hyper-parameters at their published values). Each activation is trained with n independent seeds. Each seed is scored by the median MMD^2 over 20 validation distributions. *Median* is the median score over seeds in units of 10^{-4}. *Ratio* is the ratio of medians relative to the ReLU reference, with the 95 % bootstrap confidence interval over training seeds in brackets.

Activation	n	Median $[10^{-4}]$	Ratio	95 % CI
ReLU (ref.)	30	2.91	1.00	–
GELU	20	2.80	0.96	[0.84, 1.12]
SiLU	20	3.11	1.07	[0.91, 1.31]
Tanh	20	3.89	1.34	[1.08, 1.57]

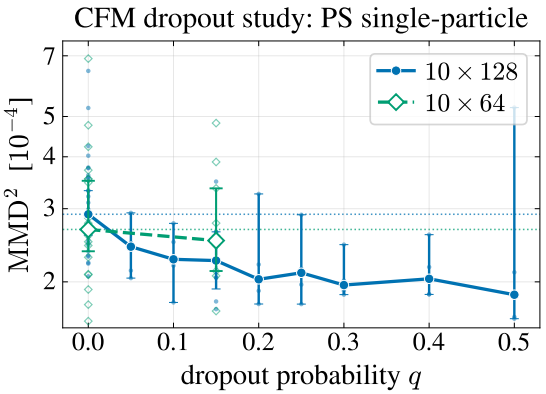

FIG. 7. Performance comparison of different dropout probabilities q for the published architecture (blue) and the smaller variant (green). Markers show the median MMD^2 over the seeds, with 95 % bootstrap confidence intervals on that median. Individual seeds are shown as faint points.

We studied the impact of different learning rate and batch size combinations on the MMD^2-performance, using the baseline architecture (depth 10, width 128). The results are shown in the left panel of Fig. 8: reducing the learning rate to 3×10^{-4} from 1×10^{-3} negatively impacts model performance across all batch sizes. Larger batch sizes also require larger learning rates to perform well. Furthermore, we evaluated the influence of overall training budget, measured in optimiser steps. We found, that the optimiser steps can be halved (122×10^3 steps instead of 244×10^3) without compromising performance, but reducing to a quarter is harmful (see right panel of Fig. 8).

TABLE II. Training hyperparameters for the published checkpoints, identical for all three tasks presented in Sec. III.

Parameter	Value
Optimiser	AdamW
LR schedule	cosine annealing
Initial learning rate	1×10^{-3}
Final learning rate	1×10^{-4}

Final evaluation. Finally, the most promising hyperparameter-combinations are evaluated on the test set for 10 seeds: depth 10, width 128, $q = 0.00$ (the published baseline); depth 10, width 128, $q = 0.15$ and depth 10, width 64, $q = 0.00$. These models are also compared to the published checkpoint, which is a single training run with the baseline configuration. The results are presented in Table IV. The baseline and the 3.8× smaller model perform equally, allowing faster training and inference. The best performance is achieved by the large model with dropout regularisation.

Appendix B: Inference

For inference, we use Dormand-Prince 5(4) [7], implemented in the torchdyn [28] and the torchdiffeq [4] package, to integrate the neural vector fields. We use absolute and relative tolerances of 1×10^{-4}. For the Gaussian probability paths, we set $\sigma = 0.01$. Table V states the inference times on the two tasks for the different methods on GPU and Table VI reports the respective values on CPU. Timings were obtained on cluster nodes, using a single NVIDIA H100 GPU (GPU measurements), and with 4 allocated CPU cores (CPU measurements).

TABLE III. Batch size and number of epochs per task and model. Batch size is given in number of particles, which are grouped per machine configuration into sets of 2000 particles. CFM on the single-particle task is the exception: particles are not grouped into sets. Where multiple values are listed, they correspond in order.

Task	Method	Training set size	Batch size	Epochs
Single-particle dynamics	CFM	200, 500, 1500	16384	2000, 2000, 500
	Hybrid-CFM	200, 500, 1500	64000 (32)	5000, 2000, 2000
Space-charge	CFM	60	64000 (32)	2000
	CA-CFM	60	64000 (32)	2000

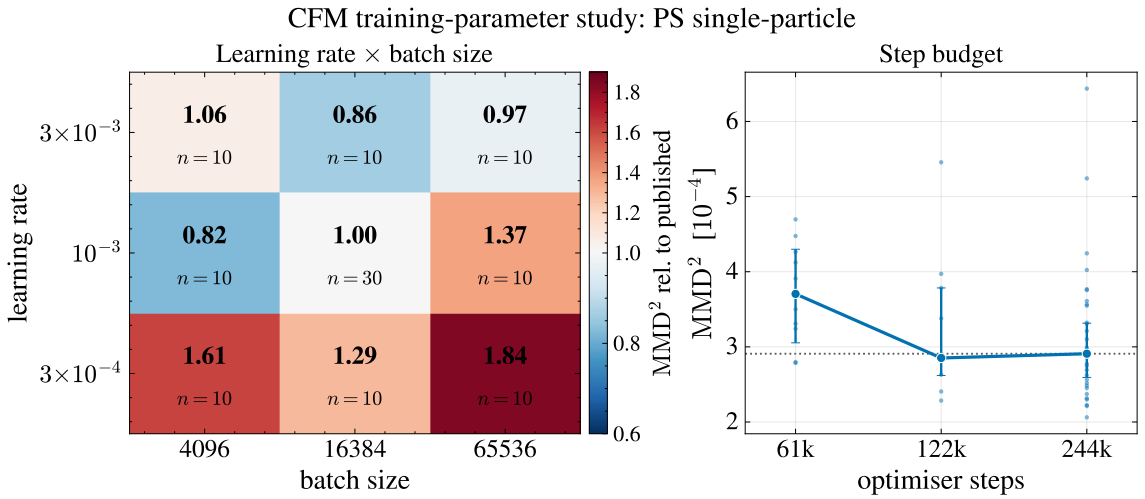

FIG. 8. Left: Median of MMD^2 over multiple seeds for different combinations of learning rate and batch size, relative to the baseline hyperparameters (learning rate $= 1 \times 10^3$ and batch size $= 16384$). Right: MMD^2 versus number of optimiser steps for the baseline architecture. Markers show the median MMD^2 over the seeds, with 95 % bootstrap confidence intervals on that median. Individual seeds are shown as faint points.

TABLE IV. Final evaluation on the held-out test set (332 distributions), performed once after all hyperparameter decisions were frozen. Each configuration is retrained with n independent seeds. Each seed is scored by the median MMD^2 over all test distributions. *Median* is the median score over seeds in units of 10^{-4}, *Params* the number of trainable weights in Millions.The published model is a single training run.

Configuration	n	Params [M]	Median [10^{-4}]
10×128, $q = 0.00$	10	0.347	2.28
10×128, $q = 0.15$	10	0.347	1.92
10×64, $q = 0.00$	10	0.092	2.38
Published model	1	0.347	1.94

TABLE V. Inference time per distribution of $N = 10\,000$ particles on GPU, averaged over the validation set. The number of auxiliary particles for Hybrid-CFM is stated in brackets in *Method*. *Params* states the number of trainable weights in units of Millions. *Model* is the duration of the network forward pass in seconds. *Aux* is the duration of the auxiliary-particle tracking with Xsuite required by Hybrid-CFM. *Total* is their sum. *Speed-up* is relative to conventional tracking with Xsuite.

Task	Method	Params [M]	Model [s]	Aux [s]	Total [s]	Speed-up
Single-particle dynamics	Xsuite	–	150.632	–	1.51×10^{2}	–
	CFM	0.347	0.041	–	4.09×10^{-2}	3.69×10^{3}
	Hybrid-CFM(10)	1.008	0.106	1.26×10^{2}	1.26×10^{2}	1.20
	Hybrid-CFM(50)	1.008	0.109	1.40×10^{2}	1.40×10^{2}	1.08
	Hybrid-CFM(100)	1.008	0.112	1.31×10^{2}	1.31×10^{2}	1.15
	Hybrid-CFM(200)	1.008	0.099	1.35×10^{2}	1.35×10^{2}	1.12
	Hybrid-CFM(500)	1.008	0.196	1.41×10^{2}	1.41×10^{2}	1.07
Space-charge	Xsuite	–	509.429	–	5.09×10^{2}	–
	CFM	0.333	0.042	–	4.22×10^{-2}	1.2×10^{4}
	CA-CFM	0.995	0.404	–	4.04×10^{-1}	1.3×10^{3}

TABLE VI. Inference time per distribution of $N = 10\,000$ particles on CPU, averaged over the validation set. The number of auxiliary particles for Hybrid-CFM is stated in brackets in *Method*. *Params* states the number of trainable weights in units of Millions. *Model* is the duration of the network forward pass in seconds. *Aux* is the duration of the auxiliary-particle tracking with Xsuite required by Hybrid-CFM. *Total* is their sum. *Speed-up* is relative to conventional tracking with Xsuite.

Task	Method	Params [M]	Model [s]	Aux [s]	Total [s]	Speed-up
Single-particle dynamics	Xsuite	–	1.19×10^{4}	–	1.19×10^{4}	–
	CFM	0.347	6.70	–	6.70	1.8×10^{3}
	Hybrid-CFM(10)	1.008	6.86	6.35×10^{1}	7.04×10^{1}	1.7×10^{2}
	Hybrid-CFM(50)	1.008	7.20	2.46×10^{2}	2.53×10^{2}	4.7×10^{1}
	Hybrid-CFM(100)	1.008	7.52	4.72×10^{2}	4.80×10^{2}	2.5×10^{1}
	Hybrid-CFM(200)	1.008	1.63×10^{1}	9.19×10^{2}	9.35×10^{2}	1.3×10^{1}
	Hybrid-CFM(500)	1.008	4.76×10^{1}	2.19×10^{3}	2.24×10^{3}	5.3
Space-charge	Xsuite	–	3.44×10^{3}	–	3.44×10^{3}	–
	CFM	0.333	6.59	–	6.59	5.2×10^{2}
	CA-CFM	0.995	9.74×10^{1}	–	9.74×10^{1}	3.5×10^{1}

[1] A. N. Angelopoulos and S. Bates. Conformal prediction: A gentle introduction. *Found. Trends Mach. Learn.*, 16 (4):494–591, Mar. 2023. ISSN 1935-8237. doi:10.1561/2200000101. URL https://doi.org/10.1561/2200000101. (↑ 10)

[2] J. Boelts, M. Deistler, M. Gloeckler, Á. Tejero-Cantero, J.-M. Lueckmann, G. Moss, et al. sbi reloaded: a toolkit for simulation-based inference workflows. *Journal of Open Source Software*, 10(108):7754, 2025. doi: 10.21105/joss.07754. (↑ 10)

[3] J. W. Burby, Q. Tang, and R. Maulik. Fast neural poincaré maps for toroidal magnetic fields. *Plasma Physics and Controlled Fusion*, 63(2):024001, dec 2020. doi:10.1088/1361-6587/abcbaa. URL https://dx.doi.org/10.1088/1361-6587/abcbaa. (↑ 2)

[4] R. T. Q. Chen. torchdiffeq, 2018. URL https://github.com/rtqichen/torchdiffeq. (↑ 13)

[5] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier. Language modeling with gated convolutional networks. In D. Precup and Y. W. Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 933–941. PMLR, 06–11 Aug 2017. URL https://proceedings.mlr.press/v70/dauphin17a.html. (↑ 5, 11)

[6] M. Deistler, J. Boelts, P. Steinbach, G. Moss, T. Moreau, M. Gloeckler, P. L. C. Rodrigues, J. Linhart, J. K. Lappalainen, B. K. Miller, P. J. Gonçalves, J.-M. Lueckmann, C. Schröder, and J. H. Macke. Simulation-based inference: A practical guide, 2025. URL https://arxiv.org/abs/2508.12939. (↑ 10)

[7] J. Dormand and P. Prince. A family of embedded runge-kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980. ISSN 0377-0427. doi: https://doi.org/10.1016/0771-050X(80)90013-3. URL https://www.sciencedirect.com/science/article/pii/0771050X80900133. (↑ 5, 13)

[8] S. Elfwing, E. Uchibe, and K. Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017. URL https://arxiv.org/abs/1702.03118. (↑ 11)

[9] G. Gallardo Romero, G. Rodríguez-Llorente, L. Magariños Rodríguez, R. Morant Navascués, N. Khvatkin Petrovsky, R. Lorenzo Ortega, and R. Gómez-Espinosa Martín. Differentiable deep learning surrogate models applied to the optimization of the ifmif-dones facility. *Particles*, 8(1), 2025. ISSN 2571-712X. doi: 10.3390/particles8010021. URL https://www.mdpi.com/2571-712X/8/1/21. (↑ 1)

[10] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773, 2012. URL http://jmlr.org/papers/v13/gretton12a.html. (↑ 4)

[11] H. Grote and F. Schmidt. MAD-X: An upgrade from MAD8. *Conf. Proc. C*, 030512:3497, 2003. (↑ 1)

[12] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90. (↑ 4, 11)

[13] D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus), 2023. URL https://arxiv.org/abs/1606.08415. (↑ 11)

[14] G. Hinton and V. Nair. Rectified linear units improve restricted boltzmann machines vinod nair. 27:807–814, 06 2010. (↑ 11)

[15] C.-K. Huang, Q. Tang, Y. K. Batygin, O. Beznosov, J. Burby, A. Kim, S. Kurennoy, T. Kwan, and H. N. Rakotoarivelo. Symplectic neural surrogate models for beam dynamics. *Journal of Physics: Conference Series*, 2687(6):062026, jan 2024. doi:10.1088/1742-6596/2687/6/062026. URL https://dx.doi.org/10.1088/1742-6596/2687/6/062026. (↑ 2)

[16] F. Huhn and F. M. Velotti. Differentiable simulations for particle tracking in accelerators: Analysis, benchmarking, and optimization. *Phys. Rev. Accel. Beams*, pages –, Oct 2025. doi:10.1103/cbds-lst1. URL https://link.aps.org/doi/10.1103/cbds-lst1. (↑ 1)

[17] G. Iadarola et al. Xsuite: An Integrated Beam Physics Simulation Framework. *JACoW*, HB2023:TUA2I1, 2024. doi:10.18429/JACoW-HB2023-TUA2I1. (↑ 1, 6)

[18] A. Ivanov and I. Agapov. Physics-based deep neural networks for beam dynamics in charged particle accelerators. *Phys. Rev. Accel. Beams*, 23:074601, Jul 2020. doi:10.1103/PhysRevAccelBeams.23.074601. URL `https://link.aps.org/doi/10.1103/PhysRevAccelBeams.23.074601`. (↑ 2)

[19] P. Jin, Z. Zhang, A. Zhu, Y. Tang, and G. E. Karniadakis. Sympnets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems. *Neural Networks*, 132:166–179, 2020. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2020.08.017. URL `https://www.sciencedirect.com/science/article/pii/S0893608020303063`. (↑ 2)

[20] J. Kaiser, C. Xu, A. Eichler, and A. Santamaria Garcia. Bridging the gap between machine learning and particle accelerator physics with high-speed, differentiable simulations. *Phys. Rev. Accel. Beams*, 27:054601, May 2024. doi:10.1103/PhysRevAccelBeams.27.054601. URL `https://link.aps.org/doi/10.1103/PhysRevAccelBeams.27.054601`. (↑ 1)

[21] K.-R. Kladny, B. Schölkopf, and M. Muehlebach. Conformal generative modeling with improved sample efficiency through sequential greedy filtering, 2025. URL `https://arxiv.org/abs/2410.01660`. (↑ 10)

[22] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6405–6416, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964. (↑ 10)

[23] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In K. Chaudhuri and R. Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 09–15 Jun 2019. URL `https://proceedings.mlr.press/v97/lee19d.html`. (↑ 11)

[24] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling, 2023. URL `https://arxiv.org/abs/2210.02747`. (↑ 2)

[25] W. J. Maddox, T. Garipov, P. Izmailov, D. Vetrov, and A. G. Wilson. *A simple baseline for Bayesian uncertainty in deep learning*. Curran Associates Inc., Red Hook, NY, USA, 2019. (↑ 10)

[26] A. B. Owen. Scrambling sobol and niederreiter–xing points. *Journal of Complexity*, 14(4):466–489, 1998. ISSN 0885-064X. doi: https://doi.org/10.1006/jcom.1998.0487. URL `https://www.sciencedirect.com/science/article/pii/S0885064X98904873`. (↑ 3)

[27] E. Parzen. On estimation of a probability density function and mode. *Ann. Math. Statist.*, 33(3):1065–1076, 1962. doi: 10.1214/aoms/1177704472. (↑ 7)

[28] M. Poli, S. Massaroli, A. Yamashita, H. Asama, J. Park, and S. Ermon. Torchdyn: Implicit models and neural numerical methods in pytorch. (↑ 13)

[29] P. Ramachandran, B. Zoph, and Q. V. Le. Searching for activation functions, 2017. URL `https://arxiv.org/abs/1710.05941`. (↑ 11)

[30] M. Remta, P. A. A. Sota, Y. Dutheil, F. Huhn, and F. Velotti. Towards tailored beam distributions for fixed target experiments at CERN. presented at IPAC'25, Taipei, Taiwan, Jun. 2025, paper TUPB018, unpublished, 2025. (↑ 1)

[31] M. Remta, A. Beck, S. Kilambi, T. Zhang, and F. Velotti. Particle tracking with physics-informed deep learning methods, 2026. URL `https://arxiv.org/abs/2608.14247`. (↑ 2, 9)

[32] M. Remta, Y. Dutheil, and F. Velotti. Flow-based surrogate models for particle tracking: datasets, Aug. 2026. URL `https://doi.org/10.5281/zenodo.21964652`. (↑ 11)

[33] M. Remta, Y. Dutheil, and F. Velotti. Flow-based surrogate models for particle tracking: trained model checkpoints, Aug. 2026. URL `https://doi.org/10.5281/zenodo.21964726`. (↑ 11)

[34] D. W. Scott. *Multivariate Density Estimation: Theory, Practice, and Visualization*. John Wiley & Sons, New York, 1992. doi: 10.1002/9780470316849. (↑ 7)

[35] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL `http://jmlr.org/papers/v15/srivastava14a.html`. (↑ 5, 11)

[36] V. Stimper, D. Liu, A. Campbell, V. Berenz, L. Ryll, B. Schölkopf, and J. M. Hernández-Lobato. normflows: A pytorch package

for normalizing flows. *Journal of Open Source Software*, 8(86):5361, 2023. doi: 10.21105/joss.05361. URL https://doi.org/10.21105/joss.05361. (↑ 11)

[37] A. Tejero-Cantero, J. Boelts, M. Deistler, J.-M. Lueckmann, C. Durkan, P. J. Gonçalves, D. S. Greenberg, and J. H. Macke. sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, 5(52):2505, 2020. doi: 10.21105/joss.02505. URL https://doi.org/10.21105/joss.02505. (↑ 10)

[38] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport, 2024. URL https://arxiv.org/abs/2302.00482. (↑ 2)

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html. (↑ 3, 11)

[40] J. Wildberger, M. Dax, S. Buchholz, S. Green, J. H. Macke, and B. Schölkopf. Flow matching for scalable simulation-based inference. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 16837–16864. Curran Associates, Inc., 2023. doi:10.52202/075280-0737. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/3663ae53ec078860bb0b9c6606e092a0-Paper-Conference.pdf. (↑ 10)